PROJECT AND TEAM INFORMATION

Project Title

Mini VCS with remote repository and smart version retention

Student / Team Information

Team Name: *Team #*	*Team Quadzz*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Rashi Sharma-* *2510382287* *rashi.poonam.sharma.07.01.* *2006@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Avni Srivastava-* *2510370007* *avnisrivastava15@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Dhanvi Sarpal-* *2510370068* *dhanvi.sarpal2007@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Aditi Sharma-* *2510370447* *sharma.aditi2008@gmail.com* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Software projects involve continuous changes to source code, documents, and other files. As a project develops, it becomes important to keep track of different versions, identify what changes were made, restore previous versions when required, and allow multiple users to work on the same project. Without an organized version-control mechanism, developers may face problems such as accidental data loss, difficulty in tracking changes, overwriting of important files, and lack of coordination among team members.

Existing version-control platforms such as Git provide powerful solutions for these problems, but their underlying concepts can be complex for beginners to understand. In addition, conventional version-control systems may retain a large number of historical versions, which can increase storage requirements when repositories contain frequently changing files.

The proposed project, ***MiniVCS****, aims to address these challenges by developing a simplified version-control and collaborative repository management system using C++. The system will allow users to create repositories, track files, commit changes, maintain version history, create branches, merge changes, compare and restore previous versions, and work with remote repositories through clone, push, and pull operations.*

The system will also provide ***public and private repositories with user authentication and access permissions****, allowing repository owners to control who can access or modify their projects. A* ***smart version-retention mechanism*** *will additionally allow users to specify how long older versions should be preserved. Versions that exceed the selected retention period can be automatically removed while maintaining the latest version.*

This project is important because it provides a practical implementation of fundamental ***Data Structures, Algorithms, Object-Oriented Programming, File Handling, Database Management, Authentication, and Networking concepts*** *in a single real-world application. It also helps demonstrate how these concepts can be combined to develop a scalable and useful software-management system while providing students with a deeper understanding of how version-control systems work internally.*

State of the Art / Current solution (1 pt)

Today, version-control and collaborative software development are mainly handled using systems such as ***Git****, along with repository-hosting platforms such as GitHub, GitLab, and Bitbucket. Git is a distributed version-control system that maintains the history of changes and allows developers to create branches, commit changes, compare versions, merge work, and restore earlier versions of a project. Each developer can work on a local copy of a repository and synchronize changes with a remote repository when required.*

Modern platforms build on Git by providing features for remote repository hosting and collaboration. They support public and private repositories, access permissions, branching workflows, code review, and tools for managing contributions from multiple developers. GitLab, for example, combines Git-based repositories with branching, collaboration, permissions, and CI/CD features.

These systems solve most of the major challenges associated with managing changing project files. However, they can be complex for beginners who want to understand how version control works internally. Our proposed ***MiniVCS*** *takes inspiration from these existing systems while focusing on a simpler implementation that demonstrates their core concepts. In addition, it introduces a* ***user-defined version-retention mechanism*** *to automatically remove outdated versions while preserving the latest version, providing a practical approach to managing repository storage.*

Project Goals and Milestones (2 pts)

*The main goal of **MiniVCS** is to develop a simplified version-control and collaborative repository management system using C++. The project aims to provide the essential features of a version-control system while helping us understand how concepts such as data structures, file handling, databases, authentication, and networking work together in a real-world application.*
The major goals of the project are to:

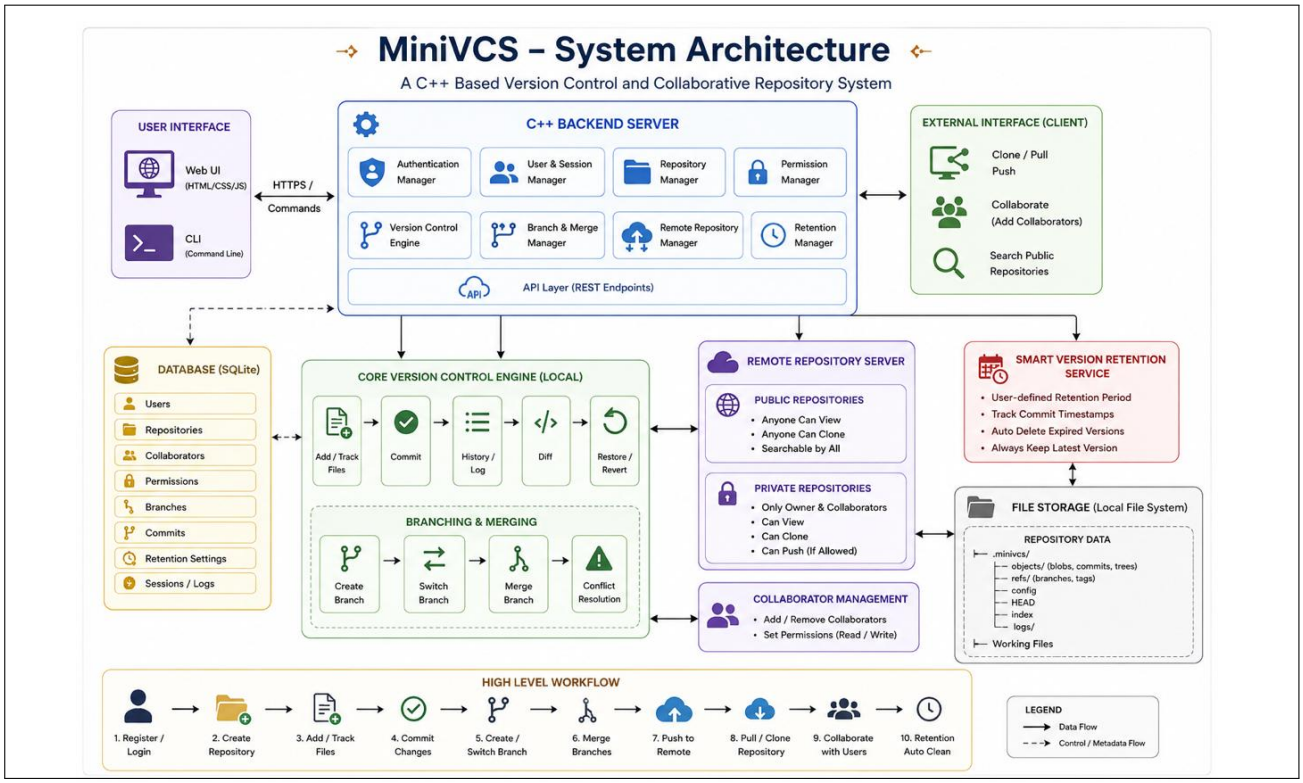
- *Develop a system for creating and managing local repositories.*
- *Implement file tracking, commits, version history, diff, and restoration of previous versions.*
- *Support branching and merging with basic conflict detection.*
- *Implement user registration, login, authentication, and repository ownership.*
- *Provide public and private repositories with collaborator-based access permissions.*
- *Enable remote repository operations such as clone, push, and pull.*
- *Use SQLite for storing user, repository, commit, branch, and permission metadata.*
- *Implement a smart version-retention mechanism that automatically removes expired older versions while preserving the latest version.*
- *Apply appropriate Data Structures and Algorithms such as hashing, trees, graphs, stacks, searching, and sorting.*

Initial and Subsequent Milestones
Milestone 1: *Study existing version-control systems and finalize system requirements and architecture.*
Milestone 2: *Implement the basic C++ repository, file tracking, commit, history, diff, and restore functionality.*
Milestone 3: *Implement branching, merging, and basic conflict detection.*
Milestone 4: *Integrate SQLite and develop user authentication, repository ownership, public/private visibility, and permissions.*
Milestone 5: *Implement remote repositories and clone, push, and pull functionality.*
Milestone 6: *Develop and integrate the smart version-retention mechanism.*
Final Milestone: *Integrate all modules, perform testing, fix issues, evaluate performance, and prepare the final documentation and demonstration.*

Project Approach (3 pts)

The proposed solution will be designed as a modular version-control system, where each major functionality is developed separately and then integrated into a single application. The core of the system will be developed in ***C++****, allowing us to implement the main version-control logic and apply Data Structures and Algorithms in a practical way.*
The first stage will focus on the local version-control engine. It will handle repository creation, file tracking, commits, version history, diff, restoration, branching, and merging. Data structures such as ***hash tables, trees, linked lists, stacks, and graphs*** *will be used to efficiently manage files, commits, branches, and their relationships.*
A ***SQLite database*** *will be used to store structured information such as users, repositories, commits, branches, collaborators, permissions, and retention settings. Actual project files and version snapshots will be stored in the file system.* ***SHA-256 hashing*** *will be used to identify file contents and detect changes.*
The system will provide a simple web-based interface using ***HTML, CSS, and JavaScript****, which will communicate with the C++ backend through HTTP-based APIs. User authentication, public/private repositories, collaborator permissions, and remote operations such as* ***clone, push, and pull*** *will then be integrated.*
Finally, a ***smart version-retention module*** *will be added to automatically identify and remove outdated versions according to the retention period selected by the user, while preserving the latest version.*
The main technologies and tools used will be ***C++, SQLite, HTML, CSS, C++ STL, SHA-256 and HTTP****. Development will be carried out incrementally, with testing performed after each module before complete system integration.*

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

*The main outcome of **MiniVCS** will be a working, simplified version-control and collaborative repository management system developed using C++. The project will demonstrate how version control, database management, authentication, networking, and Data Structures and Algorithms can be combined into one practical application.The major deliverables of the project will include:*

- *A **C++ based version-control engine** supporting repository creation, file tracking, commits, history, diff, restore, branching, and merging.*
- ***Basic merge conflict detection** for handling changes made to the same files.*
- *A **user authentication system** with registration and login functionality.*
- *Support for **public and private repositories** with owner and collaborator permissions.*
- *A **SQLite database** for storing users, repositories, commits, branches, permissions, and retention settings.*
- ***Remote repository functionality** with basic clone, push, and pull operations.*
- *A simple **web-based interface** using HTML, CSS, and JavaScript for interacting with the system.*
- *A **smart version-retention module** that removes expired older snapshots while preserving the latest version.*
- *Complete **source code, database schema, system documentation, testing results, and project demonstration**.*

The final system will provide a practical demonstration of the core ideas behind modern version-control and collaborative development tools in a simpler and educational form.

Assumptions

The project assumes that users have basic computer knowledge and access to a system capable of running C++ applications. We assume that repositories will mainly contain text-based source-code files and that the number of users and repositories will be suitable for a college-level project. The system will use SQLite for storing metadata and the local file system for storing actual files and versions. We assume a stable network connection is available for remote operations such as clone, push, and pull. The retention period is assumed to be defined by the repository owner, and the latest version will always be preserved.

References

1. *Git – Git documentation and basic version control concepts https://git-scm.com/docs*
2. *GitHub Docs – Repositories, branches and collaboration https://docs.github.com/*
3. *SQLite – Official SQLite documentation https://www.sqlite.org/docs.html*
4. *C++ Reference – C++ language and STL reference https://en.cppreference.com/*
5. *W3Schools – HTML, CSS and JavaScript basics https://www.w3schools.com/*
6. *GeeksforGeeks – Data Structures, Algorithms and C++ concepts https://www.geeksforgeeks.org/*